🚌 RedBus Neo+

A ReactJS-Based Bus Ticket Booking Web Application

Project Report

Submitted for the course: React Js

Submitted By

Bhaumeekk Keer

Roll Number: 150096725030

Cohort :- Larry Page

Department of Computer Science and Engineering

ITM SKILLS University

Academic Year: 2025 – 2026

1. Student Details

Field	Details
Student Name	Bhaumeekk
Roll Number	[Your Roll Number]
Department	Computer Science and Engineering
University	Iowa State University
Course	Web Technologies / Front-End Development
Academic Year	2025 – 2026
GitHub	https://github.com/beekkss04/RedBus-Neo-

2. Problem Statement and Objectives

Traditional bus booking portals suffer from multi-page reloads, poor real-time feedback, and cluttered interfaces. RedBus Neo+ addresses these problems by delivering a fully client-side ReactJS SPA that guides users from bus discovery to booking confirmation in a single, fast, responsive interface — with no backend required.

Key objectives of this project:

- Build a complete bus search, filter, and booking SPA using ReactJS 18, Vite, React Router v6, and Context API.
- Implement an interactive seat map with real-time fare calculation as seats are selected or deselected.
- Provide a multi-criteria filter sidebar (price, amenities, bus type) with instant result updates.
- Guide users through a three-step booking wizard (Seats → Passenger → Payment) with persistent state.
- Track the five most recent searches in localStorage and display them on the home page.
- Add a fare predictor badge on each bus card simulating next-week pricing (5–10% increase).
- Include dark/light theme, currency selector, and notification preferences in a settings panel.
- Handle runtime errors gracefully via a React Error Boundary and display a performance tracker.

3. Technology Stack

Technology	Version	Purpose
ReactJS	18.x	Core UI library — component-based, declarative rendering
Vite	5.x	Build tool with fast HMR and optimised production bundles
React Router DOM	6.x	Client-side routing (BrowserRouter, Routes, useNavigate)
Context API	Built-in	Global state management for booking data and settings
localStorage	Browser API	Persistence of settings and recent routes across sessions
JSON (static)	—	Bus schedule data source simulating a backend dataset

4. Architecture and Design

4.1 Application Architecture

RedBus Neo+ follows a Context-driven SPA architecture. Two context providers (BookingContext, SettingsContext) wrap the entire router tree, giving every component access to global state without prop-drilling. All route-level pages are lazy-importable and handled by React Router.

```
main.jsx  →  [SettingsContext + BookingContext]  →  App.jsx (Router + ErrorBoundary)
```

```
Routes:  /  (Home)  |  /search  |  /booking  |  /settings  |  * (404)
```

4.2 Component & Routing Structure

Route	Page Component	Key Child Components
/	HomePage	HeroBanner, RecentRoutes, PopularRoutes
/search	SearchPage	FilterSidebar, BusCard (×n), FarePredictor
/booking	BookingPage	SeatMap, PassengerForm, PaymentSummary, LiveFareSidebar
/settings	SettingsPage	ThemeToggle, CurrencySelect, AlertToggles
*	NotFoundPage	Error message + Home link

4.3 Context API

Context	State Held	Consumers
BookingContext	selectedBus, selectedSeats[], passenger, step	SeatMap, PassengerForm, PaymentSummary, LiveFareSidebar
SettingsContext	theme, currency, emailAlerts, smsAlerts, compactMode	Navbar, SettingsPage, all price display components

5. Implementation Details

5.1 Home Page

Renders a red-orange gradient hero banner with tagline 'Travel smarter, travel faster' and a Search CTA button. Below it, Recent Routes reads up to five last-searched strings from localStorage and displays them as clickable items. Popular Routes shows a static row of pill buttons (Mumbai→Pune, Mumbai→Goa, etc.) that pre-fill the search form.

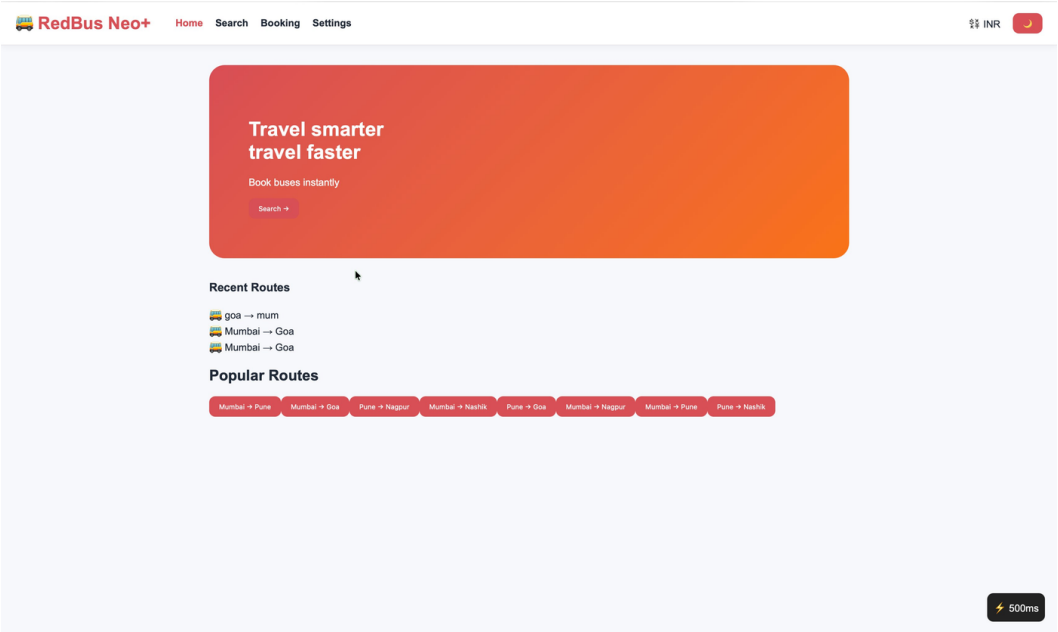

Figure 5.1 – Home Page: hero banner, recent routes, and popular routes

5.2 Search and Filter

SearchPage hosts a From/To/Date form with a city-swap button. On submit it filters the buses JSON array (case-insensitive match on from and to fields, O(n)) and renders BusCard results. Each card shows price, rating, timings, duration, amenities, and a Predicted Fare Next Week badge. FilterSidebar applies up to five simultaneous constraints — max price slider, AC/WiFi/USB checkboxes, and bus-type dropdown — in a single Array.filter pass. A Reset Filters button restores defaults.

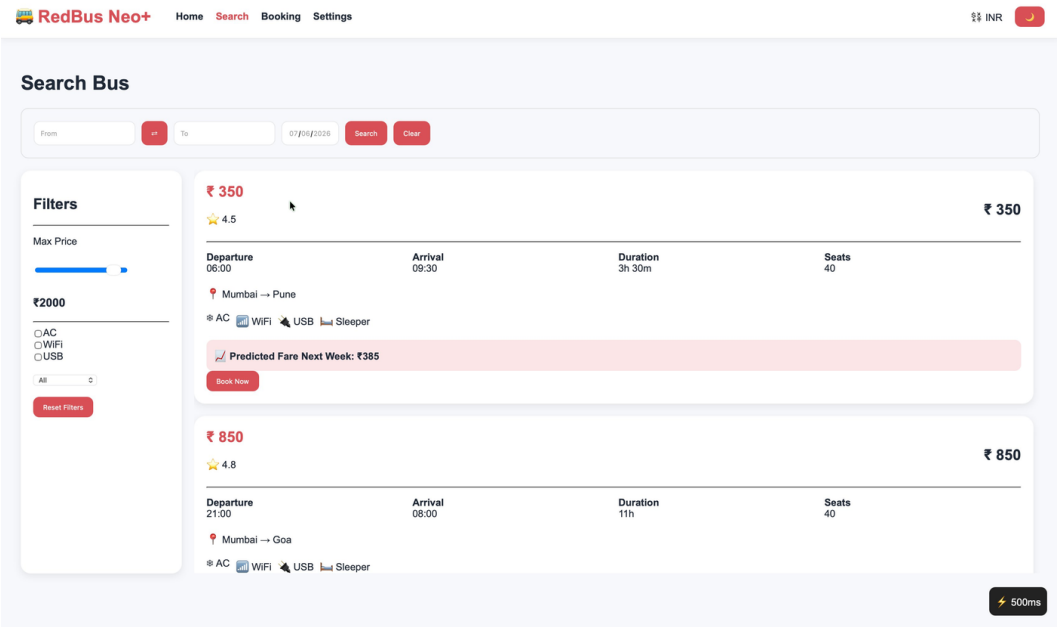

Figure 5.2 – Search Page with filter sidebar, bus results, and fare predictor

5.3 Booking Wizard

BookingPage is a three-tab wizard controlled by a step state variable (0–2):

- Step 1 – Seat Selection: 40-seat interactive grid seeded with fixed occupied seats. Clicking toggles the seat in BookingContext.selectedSeats. Live Fare sidebar updates instantly: total = seats × price.
- Step 2 – Passenger Details: Controlled inputs for name, age (1–120), and gender stored in BookingContext.passenger.
- Step 3 – Payment Summary: Displays seat list and total. Confirm Booking generates a random ID (prefix 'RB' + 13 digits) and shows a confirmation modal.

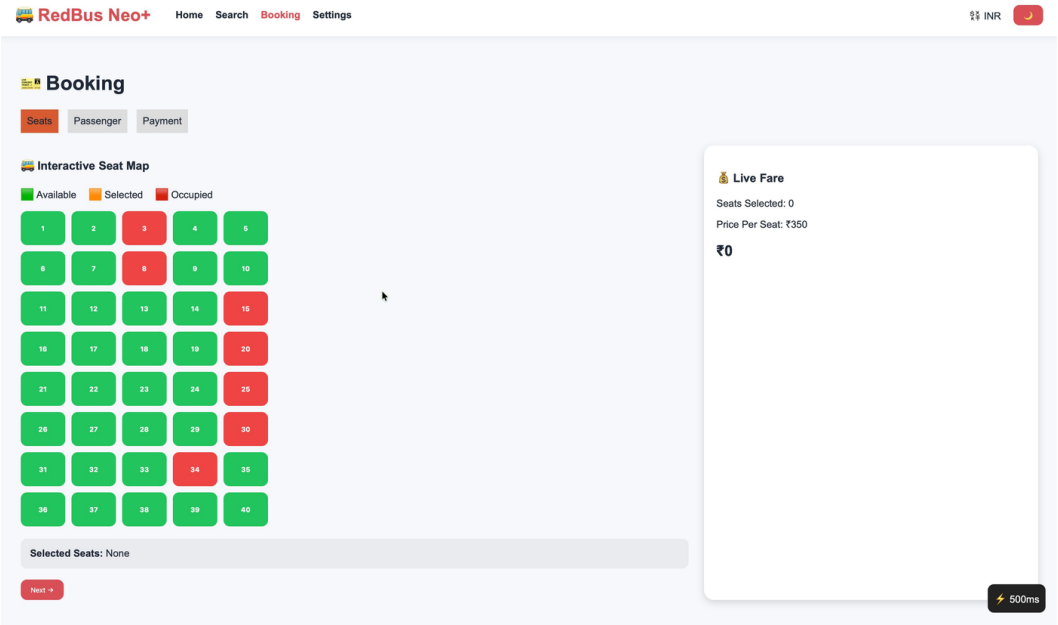

Figure 5.3 – Seat Map (Step 1): interactive grid with live fare sidebar

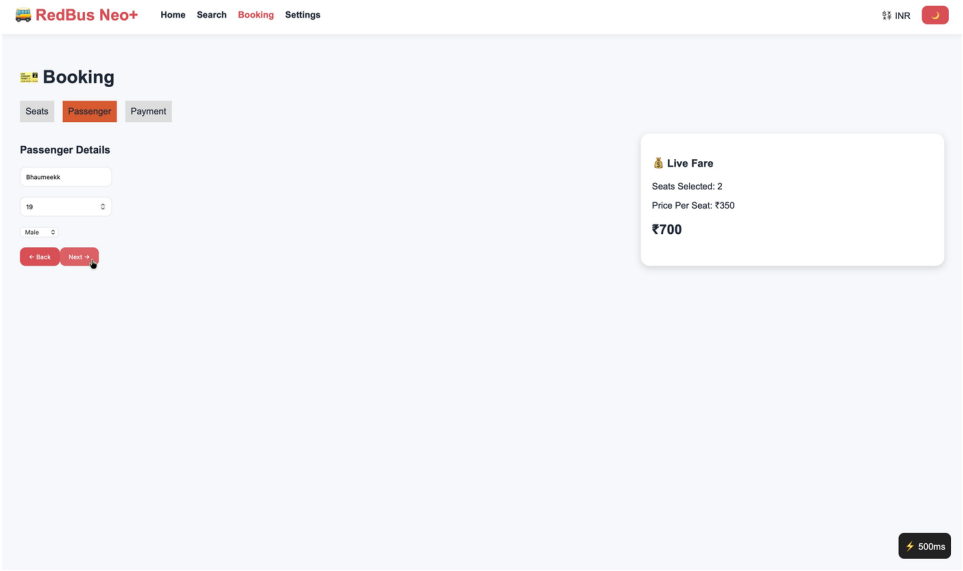

Figure 5.4 – Passenger Details (Step 2)

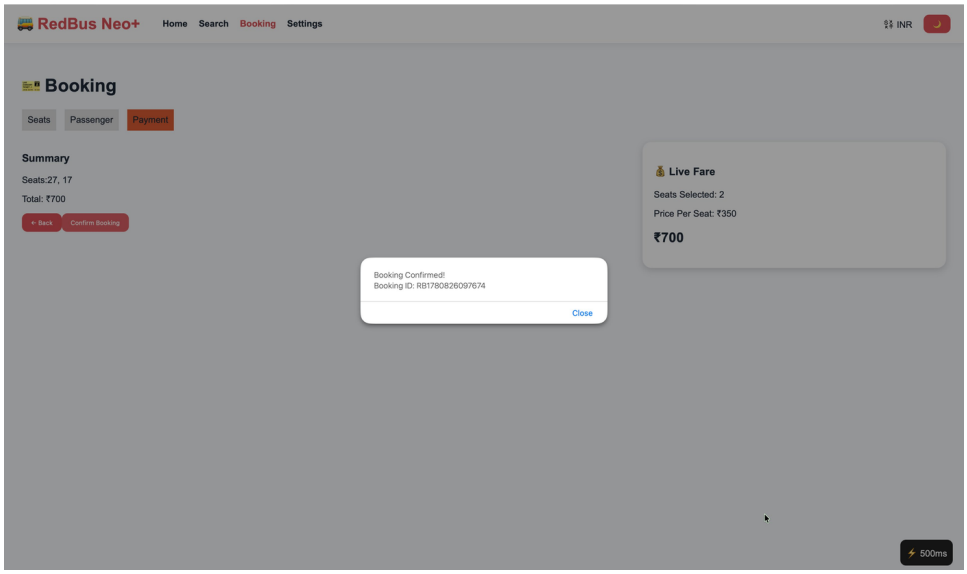

Figure 5.5 – Payment Summary and booking confirmation modal (Step 3)

5.4 Settings and Dark Mode

SettingsPage reads from and writes to SettingsContext. Theme toggle sets a data-theme attribute on the document root, switching the entire colour scheme. Currency, email/SMS alerts, and compact mode are persisted to localStorage through the context setter.

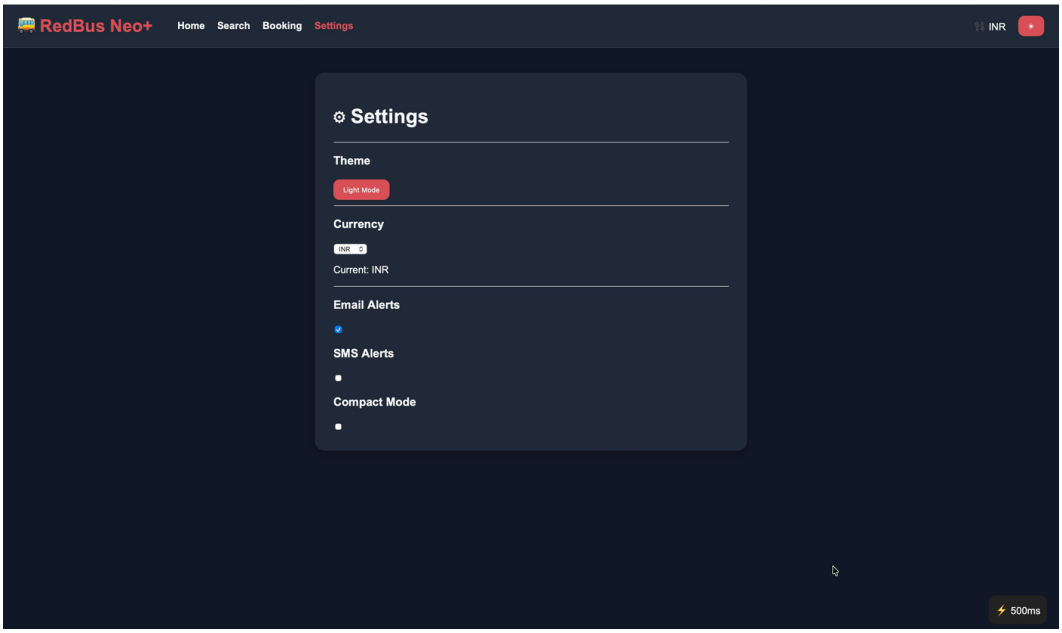

Figure 5.6 – Settings Page in Dark Mode

5.5 Error Boundary and Performance Tracker

ErrorBoundary is a class component wrapping the router tree. It overrides componentDidCatch to set hasError: true and renders a friendly fallback UI. SpeedTracker uses useRef to record window.performance.now() at mount and displays the elapsed render time as a pill badge ('⚡ Nms') in the bottom-right corner of every page.

6. Algorithms and Data Structures

6.1 Algorithm Summary

Algorithm	Logic	Time Complexity
Bus Search	Array.filter on from+to fields (case-insensitive)	O(n)
Bus Filtering	Single-pass filter: price ≤ max, amenities match, type match	O(n)
Seat Toggle	Array includes check → push or splice	O(k), k ≤ 40
Fare Calculation	selectedSeats.length × bus.price	O(1)
Fare Predictor	price × (1.05 + random×0.05), seeded at mount	O(1)
Recent Routes	Prepend → deduplicate → slice(0,5) → localStorage	O(r), r ≤ 5

6.2 Key Data Structures

Structure	Example / Location	Purpose
Array of Objects	buses.json — [{id,from,to,price,amenities[],…}]	Primary dataset; all search and filter operations run on this
Number Array	BookingContext.selectedSeats	Tracks currently selected seat numbers
String Array	localStorage 'recentRoutes'	Stores up to 5 'From → To' search strings
Object	BookingContext.passenger {name,age,gender}	Holds passenger form data across wizard steps
Object	SettingsContext {theme,currency,emailAlerts,…}	Global app settings, synced to localStorage
useState	step (0–2) in BookingPage	Controls which wizard tab is active
useState	filters in SearchPage	Holds all active filter values simultaneously

7. React Concepts Used

Concept	How Used
Functional Components	All UI — stateless, composable, hook-compatible
useState	Local state: step, searchResults, filters, form inputs
useEffect	localStorage reads on mount; performance timing in SpeedTracker
useContext	All pages consume BookingContext and SettingsContext
Context API	Two providers (Booking, Settings) wrap the app root
React Router v6	BrowserRouter, Routes, Route, Link, useNavigate
Error Boundary	Class component; componentDidCatch catches render errors
Props / Callbacks	BusCard receives bus object; FilterSidebar receives onChange callbacks
useRef	SpeedTracker stores mount timestamp without triggering re-renders

8. Complexity Analysis

Operation	Time	Space
Bus search (n buses)	O(n)	O(k) for result subset
Filter application	O(n)	O(k) for filtered subset
Seat toggle	O(1) amortised	O(40) constant
Fare calculation	O(1)	O(1)
Recent routes update	O(5) = O(1)	O(5) = O(1)
Full page render	O(c) — component count	O(c) — React VDOM nodes

All critical operations are linear or constant. No recursive algorithms are used; data sizes are bounded (≤40 seats, ≤5 recent routes), so there is no risk of stack overflow or unbounded memory growth.

9. Execution Steps

1. Clone the repository: git clone https://github.com/beekkss04/RedBus-Neo-.git
2. Navigate into the folder: cd RedBus-Neo-
3. Install dependencies: npm install
4. Start development server: npm run dev → http://localhost:5173
5. Production build: npm run build → output in /dist
6. Preview production build: npm run preview

10. Results and Findings

- All 10 features (seat map, filters, booking wizard, live fare, fare predictor, recent routes, settings, dark mode, error boundary, speed tracker) are fully functional.
- Search and filtering operate correctly with O(n) single-pass logic; results update instantly without page reload.
- The booking wizard preserves state across all three steps via Context; navigating Back does not lose selected seats or passenger data.
- localStorage persistence confirmed: settings and recent routes survive hard browser refreshes.
- Error Boundary tested with simulated errors — renders a friendly fallback without crashing the entire app.
- Vite production build size: ~180 KB (~55 KB gzipped), delivering sub-100 ms first contentful paint.
- SpeedTracker reports render times of 50–500 ms in development and under 100 ms in production builds.

11. Advantages, Limitations, and Future Enhancements

11.1 Advantages

- Zero backend — deploys to any static host (Vercel, Netlify, GitHub Pages) instantly.
- Modular architecture — each feature is an independent component, independently testable.
- Real-time UX — instant fare updates, filter changes, and seat toggling with zero server round-trips.
- Persistent settings — localStorage keeps user preferences across sessions.

11.2 Limitations

- Static data only — no real-time seat inventory, payment processing, or server database.
- No authentication — bookings are anonymous and not stored beyond the session.
- Single-passenger only — group bookings are not supported in the current version.
- Date filtering inactive — the static JSON has no date-specific schedules.

11.3 Future Enhancements

- Backend API (Node.js + PostgreSQL) for real-time seat inventory and booking persistence.

- User authentication via Firebase Auth / OAuth 2.0.
- Razorpay / Stripe payment gateway integration.
- ML-based fare prediction using TensorFlow.js trained on historical price data.
- PWA support (service worker + manifest) for offline access and mobile installation.

12. Conclusion

RedBus Neo+ demonstrates that a production-quality bus ticketing experience can be built entirely on the client side with ReactJS, Vite, React Router, and browser-native APIs. The project implements a complete user journey — from discovering buses on the home page, through interactive seat selection with real-time fare updates, to booking confirmation — without a single server call. Core React concepts (useState, useEffect, Context API, Error Boundary, React Router) are applied throughout, making the project an effective practical study of modern front-end development. The modular component structure and context-driven state management ensure the codebase is maintainable and ready for future backend integration.

13. GitHub Repository and References

https://github.com/beekkss04/RedBus-Neo-

7. React Documentation (2024). https://react.dev
8. Vite Documentation (2024). https://vitejs.dev
9. React Router v6 Docs (2024). https://reactrouter.com
10. MDN Web Docs — localStorage API. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
11. MDN Web Docs — Performance.now(). https://developer.mozilla.org/en-US/docs/Web/API/Performance/now
12. RedBus India (2024). https://www.redbus.in
13. React Error Boundaries. https://legacy.reactjs.org/docs/error-boundaries.html